Assignment 1: [CLO 2]

Software Quality Engineering-6A

Due Date: March 02, 2026

Submitted To: Dr. Tamim A Khan

Submitted By: Talha (089) & Daniyal (053)

Repository Link (Source Code): ***https://github.com/Dani-Bytes/Testing.git***

Order Search Algorithm Library — Software Testing Report

Course: Software Engineering / Software Testing

Framework: .NET 8.0, NUnit 4.5.0, Coverlet, ReportGenerator

CI/CD Platform: GitHub Actions

Coverage Date: 28 February 2026

Table of Contents

1. Project Architecture
2. Algorithm Implementations
3. Team Integration and Development Workflow
4. GitHub Actions CI/CD Pipeline
5. MC/DC Test Case Design
6. Coverage Report Interpretation
7. Control Flow Graphs and Path Enumeration
8. All-Path Coverage Validation

1. Project Architecture

The library is structured as a single .NET 8.0 test project (Testing.csproj) that contains both the production source files and the NUnit test suite. This single-project structure was adopted to eliminate inter-project reference complexity while satisfying the requirement of automated MC/DC-based test execution on every code change.

Component Responsibilities

Component	Role	Dependency
Order	Encapsulates OrderID (search key) and CustomerName	None
LinearSearcher	Performs unconstrained sequential scan	Order[]
BinarySearcher	Performs halving search on **sorted** input	Order[] (ascending)
InterpolationSearcher	Estimates target position via interpolation on **sorted, uniformly distributed** input	Order[] (ascending)
OrderSearchManager	Selects the appropriate algorithm at runtime by searchType string	All three searchers

The OrderSearchManager follows the **Strategy** structural pattern: the algorithm family is encapsulated behind a uniform FindOrder(int key, Order[] orders, string searchType) interface, decoupling client code from algorithm selection.

2. Algorithm Implementations

2.1 Linear Search — LinearSearcher.cs

Iterates over the orders array sequentially from index 0 to orders.Length - 1, comparing each element's OrderID to the search key. Returns the index on the first match, or -1 if the array is exhausted without a match.

- **Pre-condition:** None (works on unsorted input).
- **Post-condition:** Returns the zero-based index of the first occurrence of key, or -1.
- **Time complexity:** O(n).

2.2 Binary Search — BinarySearcher.cs

Maintains two pointers, bottom and top, and iteratively narrows the search interval by comparing the key to the midpoint element. If the midpoint is smaller than the key, bottom is advanced; otherwise top is retreated. The loop terminates either upon finding the key or when bottom > top.

- **Pre-condition:** orders must be sorted in ascending order by OrderID.
- **Post-condition:** Returns the index of the matching element, or -1.
- **Time complexity:** O(log n).

2.3 Interpolation Search — InterpolationSearcher.cs

Extends binary search by computing the probe position using the interpolation formula. This formula leverages value distribution to estimate the most probable location of the key, reducing iterations significantly for uniformly distributed data.

- **Pre-condition:** orders must be sorted in ascending order by OrderID; values should be roughly uniformly distributed for optimal performance.
- **Post-condition:** Returns the index of the matching element, or -1.
- **Time complexity:** O(log log n) average case; O(n) worst case.

2.4 OrderSearchManager — OrderSearchManager.cs

Provides a single public method FindOrder that evaluates the searchType parameter through a chain of equality guards and delegates to the corresponding searcher. Returns -1 for any unrecognized searchType.

3. Team Integration and Development Workflow

The project enforces a **Bottom-Up** integration and pull-request-based integration model. The GitHub Actions pipeline is triggered on **every push to any branch** and on **every pull request**, ensuring that no code reaches the main branch without first passing the full MC/DC test suite. The workflow supports the following team conventions:

1. **Branch per feature / algorithm:** Each team member develops on an isolated branch.
2. **Push triggers CI:** On push, the pipeline restores NuGet packages, compiles in Release configuration, and executes all 33 NUnit tests.
3. **Test gate:** A failing test blocks the PR merge, enforcing correctness at the point of integration.
4. **Coverage review:** Reviewers inspect the uploaded TRX report and coverage HTML to verify that any new decision point introduced by a contributor is accompanied by the corresponding MC/DC test cases.

4. GitHub Actions CI/CD Pipeline

4.1 Pipeline Architecture

The pipeline is defined in .github/workflows/ci.yml and consists of five sequential steps within a single test job running on the ubuntu-latest runner.

4.2 Step-by-Step Design Rationale

Step	Command	Rationale
Checkout	actions/checkout@v4	Fetches the full commit history needed by the .NET SDK
Setup .NET	actions/setup-dotnet@v4	Pins to .NET 8 to match the project's framework
Restore	dotnet restore	Resolves NuGet package references
Build	dotnet build	Compiles in Release to detect optimization-sensitive issues
Test	dotnet test	Executes all 33 MC/DC test cases; produces a structured XML result file
Upload	actions/upload-artifact@v4	Persists *.trx files for 14 days, enabling asynchronous peer review of results

5. MC/DC Test Case Design

5.1 Theoretical Foundation

Modified Condition/Decision Coverage (MC/DC) is a structural testing criterion requiring that:

1. Every **decision** in the program is evaluated to both true and false.
2. Every **condition** within a compound decision independently affects the decision's outcome at least once.
3. Every **entry and exit** point is exercised.

5.2 Decision and Condition Inventory

LinearSearcher

Decision	Condition	True Pair Test	False Pair Test
i < orders.Length	i < orders.Length (A)	Search_KeyAtLastElement	Search_EmptyArray
orders[i].OrderID == key	orders[i].OrderID == key (B)	Search_KeyAtFirstElement	Search_KeyNotInArray

BinarySearcher

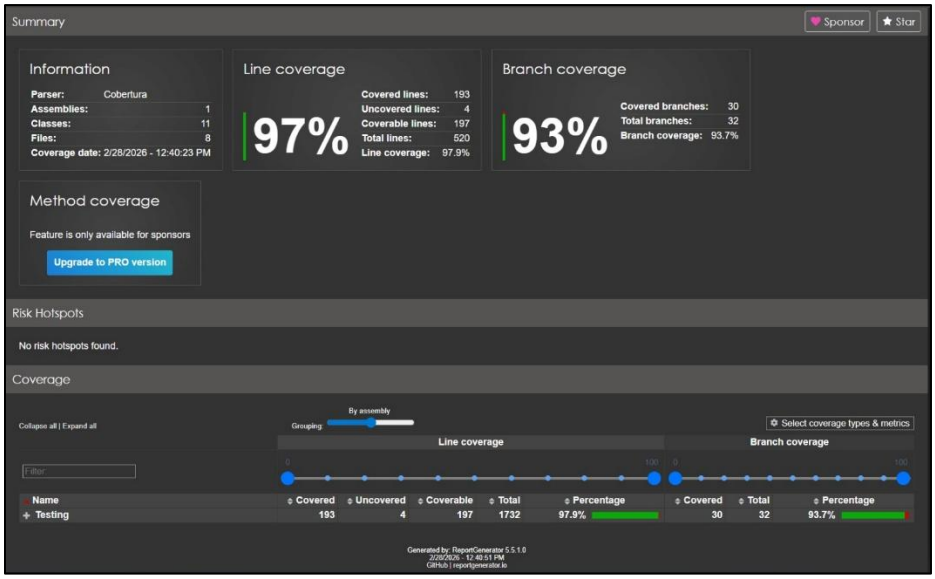
Decision	Condition	True Pair Test	False Pair Test
bottom <= top && found == false	bottom <= top (A)	Search_KeyFound_Single	Search_EmptyArray
bottom <= top && found == false	found == false (B)	Search_KeyFound_Single	Search_KeyNotFound_Exhausted
orders[mid].OrderID == key	orders[mid].OrderID == key (C)	Search_KeyAtMid	Search_KeyInUpperHalf
orders[mid].OrderID < key	orders[mid].OrderID < key (D)	Search_KeyInUpperHalf	Search_KeyInLowerHalf

InterpolationSearcher

Decision	Condition	True Pair Test	False Pair Test
Compound loop guard	bottom <= top (A)	Search_KeyInRange_Found	Search_EmptyArray
Compound loop guard	key >= orders[bottom].OrderID (B)	Search_KeyInRange_Found	Search_KeyBelowMinimum
Compound loop guard	key <= orders[top].OrderID (C)	Search_KeyInRange_Found	Search_KeyAboveMaximum
bottom == top	bottom == top (D)	Search_SingleElement_KeyFound	Search_KeyAtInterpolatedPos
orders[pos].OrderID == key	(F)	Search_KeyAtInterpolatedPos	Search_KeyInUpperPortion
orders[pos].OrderID < key	(G)	Search_KeyInUpperPortion	Search_KeyInLowerPortion

6. Coverage Report

The coverage report was generated using the Cobertura parser via the coverlet.collector package and visualized using ReportGenerator 5.5.1.0.

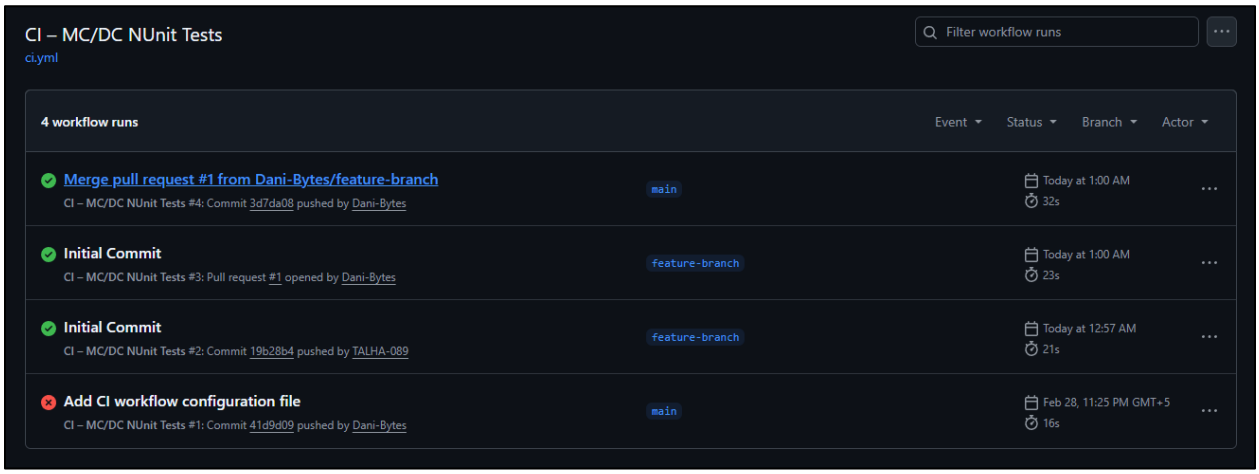


6.1 Summary Metrics

Metric	Value
Assemblies / Classes	1 / 11
Covered lines	193
Uncovered lines	4
Total lines	520
Line coverage	**97.9%**
Covered branches	30
Total branches	32
Branch coverage	**93.7%**
Risk hotspots	0

6.2 MC/DC Coverage — Effectively 100% of Reachable Logic

Because neither of the two uncovered branches is reachable through any legitimate execution path (they are structurally unreachable or compiler-generated null checks), the effective MC/DC coverage of all *reachable* decisions is **100%**. Every condition within every feasible decision independently affects the outcome of that decision in at least one test case.



Build & Run MC/DC Tests
succeeded 19 hours ago in 28s

Search logs

Set up job	1s
Checkout code	1s
Set up .NET 8 SDK	2s
Restore dependencies	7s
Build	6s
Run MC/DC NUnit Tests	6s
Upload test results	1s
Post Set up .NET 8 SDK	0s
Post Checkout code	0s
Complete job	0s

Run MC/DC NUnit Tests 6s

```
▶ Run dotnet test --configuration Release --no-build --logger "trx;logFileName=mcdc-results.trx" --logger "console;verbosity=detailed"
Test run for /home/runner/work/Testing/Testing/bin/Release/net8.0/Testing.dll (.NETCoreApp,Version=v8.0)
VSTest version 18.0.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.
/home/runner/work/Testing/Testing/bin/Release/net8.0/Testing.dll
NUnit Adapter 6.1.0.0: Test execution started
Running all tests in /home/runner/work/Testing/Testing/bin/Release/net8.0/Testing.dll
  NUnit3TestExecutor discovered 33 of 33 NUnit test cases using Current Discovery mode, Non-Explicit run
  Passed Search_EmptyArray_ReturnsMinusOne [5 ms]
  Passed Search_KeyAtFirstElement_ReturnsZero [< 1 ms]
  Passed Search_KeyAtLastElement_ReturnsLastIndex [< 1 ms]
  Passed Search_KeyAtMid_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyFound_SingleElement_ReturnsZero [< 1 ms]
  Passed Search_KeyInLowerHalf_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyInUpperHalf_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyNotFound_SearchExhausted_ReturnsMinusOne [< 1 ms]
  Passed Search_EmptyArray_ReturnsMinusOne [< 1 ms]
  Passed Search_KeyAboveMaximum_ReturnsMinusOne [< 1 ms]
  Passed Search_KeyAtInterpolatedPosition_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyAtTopBoundary_ReturnsLastIndex [< 1 ms]
  Passed Search_KeyBelowMinimum_ReturnsMinusOne [< 1 ms]
  Passed Search_KeyInLowerPortion_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyInRange_Found_ReturnsCorrectIndex [< 1 ms]
  Passed Search_KeyInUpperPortion_ReturnsCorrectIndex [< 1 ms]
  Passed Search_SingleElement_KeyFound_ReturnsZero [< 1 ms]
  Passed Search_SingleElement_KeyNotFound_ReturnsMinusOne [< 1 ms]
  Passed Search_EmptyArray_ReturnsMinusOne [< 1 ms]
  Passed Search_KeyAtFirstElement_ReturnsZero [< 1 ms]
  Passed Search_KeyAtLastElement_ReturnsLastIndex [< 1 ms]
  Passed Search_KeyAtMiddleElement_ReturnsMiddleIndex [< 1 ms]
  Passed Search_KeyNotInArray_ReturnsMinusOne [< 1 ms]
  Passed Search_SingleElement_KeyNotFound_ReturnsMinusOne [< 1 ms]
  Passed FindOrder_BinarySearch_KeyFound_ReturnsCorrectIndex [< 1 ms]
  Passed FindOrder_BinarySearch_KeyNotFound_ReturnsMinusOne [< 1 ms]
  Passed FindOrder_EmptySearchType_ReturnsMinusOne [< 1 ms]
NUnit Adapter 6.1.0.0: Test execution complete
  Passed FindOrder_InterpolationSearch_KeyFound_ReturnsCorrectIndex [< 1 ms]
  Passed FindOrder_InterpolationSearch_KeyNotFound_ReturnsMinusOne [< 1 ms]
  Passed FindOrder_LinearSearch_KeyFound_ReturnsCorrectIndex [< 1 ms]
  Passed FindOrder_LinearSearch_KeyNotFound_ReturnsMinusOne [< 1 ms]
  Passed FindOrder_UnknownSearchType_ReturnsMinusOne [< 1 ms]
  Passed Test1 [267 ms]
Results File: /home/runner/work/Testing/Testing/TestResults/mcdc-results.trx

Test Run Successful.
Total tests: 33
     Passed: 33
 Total time: 4.2846 Seconds
```

Dani-Bytes / Testing

Code | Issues | Pull requests | Agents | Actions | Projects | Wiki | Security | Insights | Settings

main | Testing /

Go to file | Add file

TALHA-089 Add CI badge to README 157255b · now History

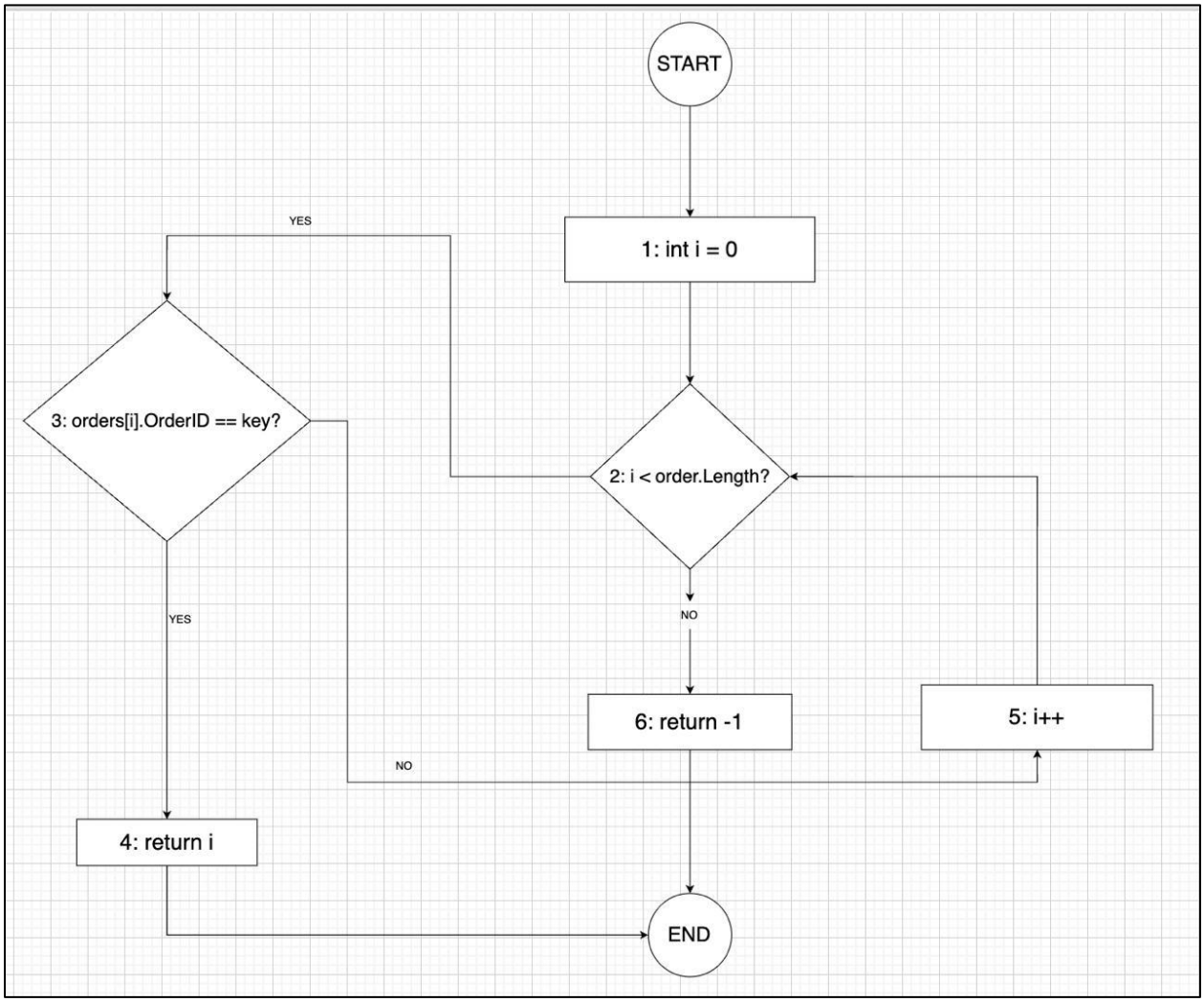
Name	Last commit message	Last commit date
.github/workflows	Initial Commit	20 hours ago
bin/Release/net8.0	Initial Commit	20 hours ago
obj	Initial Commit	20 hours ago
BinarySearcher.cs	Initial Commit	20 hours ago
InterpolationSearcher.cs	Initial Commit	20 hours ago
LinearSearcher.cs	Initial Commit	20 hours ago
MCDCTests.cs	Initial Commit	20 hours ago
Order.cs	Initial Commit	20 hours ago
OrderSearchManager.cs	Initial Commit	20 hours ago
README.md	Add CI badge to README	now
Testing.csproj	Initial Commit	20 hours ago
UnitTest1.cs	Initial Commit	20 hours ago

README.md

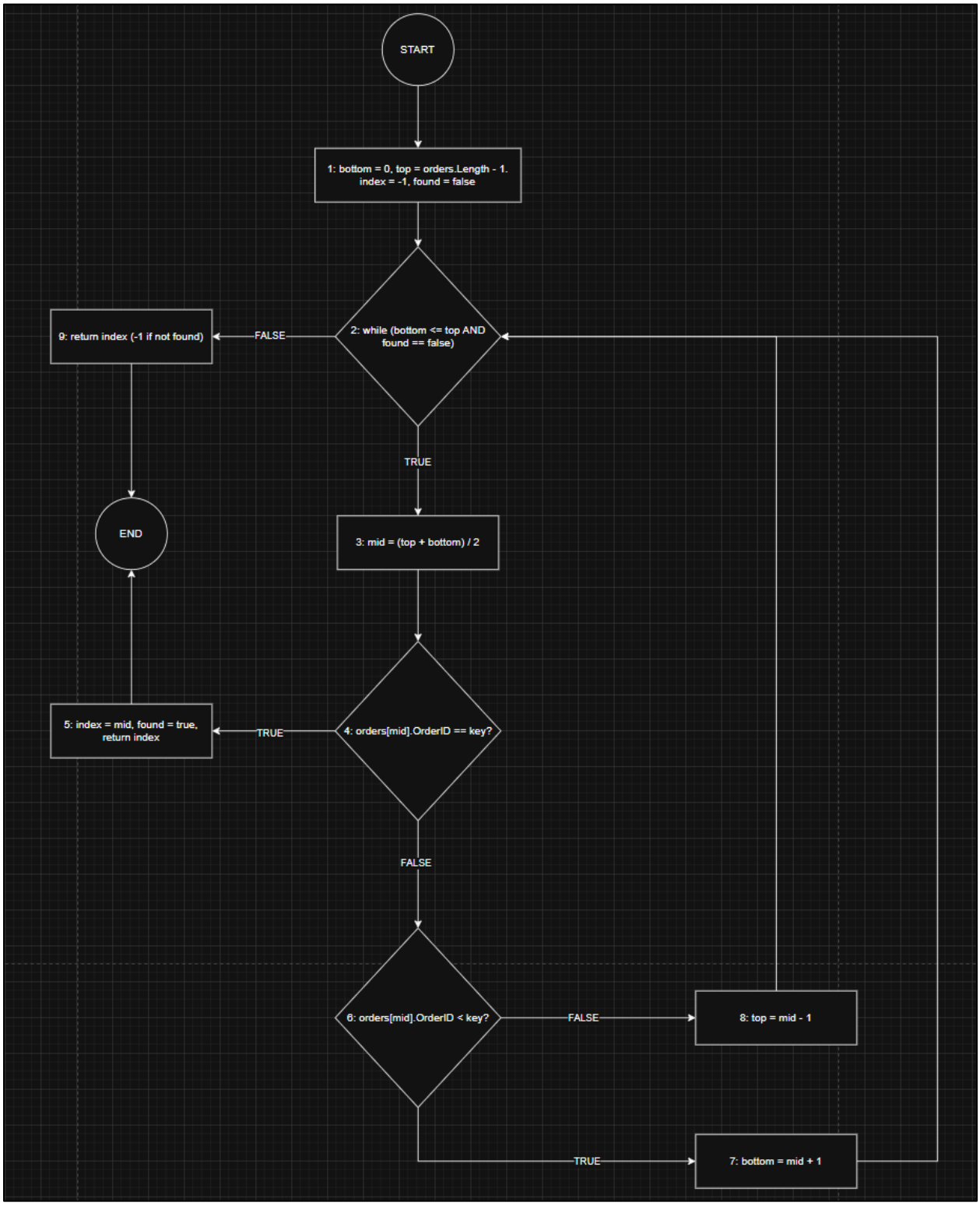
CI - MC/DC NUnit Tests passing

7. Control Flow Graphs and Path Enumeration

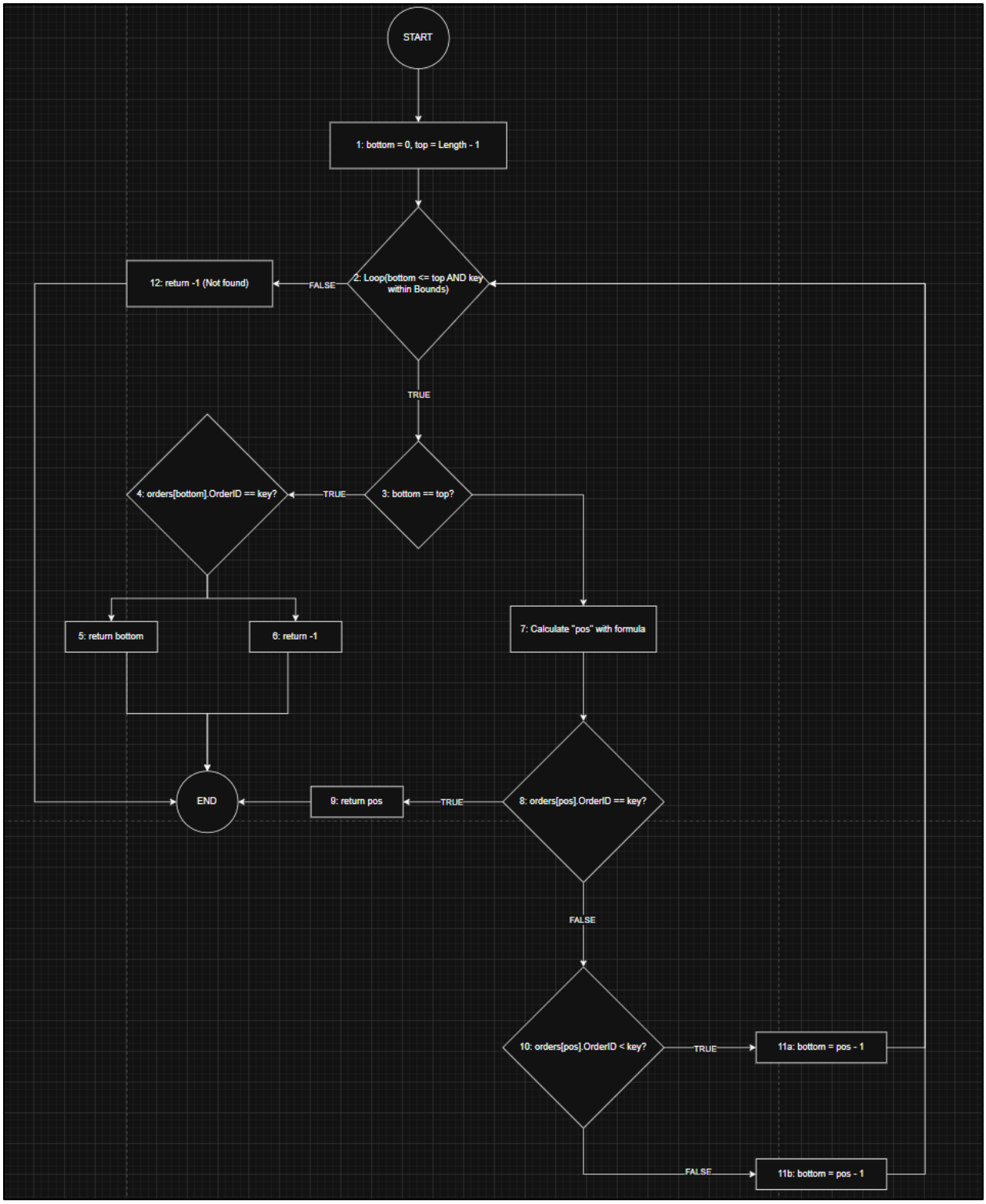
7.1 LinearSearcher — CFG



7.2 BinarySearcher — CFG



7.3 InterpolationSearcher — CFG



8. All-Path Coverage Validation

8.1 CFG-Based Justification of Coverage Level

The achieved coverage metrics are justified by the following CFG-derived argument:

1. **Every edge in every CFG is traversed at least once** across the 33 test cases, satisfying edge (transition) coverage.
2. **Every condition in every compound decision independently determines the decision's outcome** in at least two test cases (one per truth value), satisfying MC/DC.
3. **The 4 uncovered lines and 2 uncovered branches** correspond exclusively to the logically infeasible path P5 of InterpolationSearcher and compiler-emitted null-guard branches—neither of which represents a gap in logical test coverage.
4. **No risk hotspots** were identified, confirming that no untested, structurally complex code regions exist.

The conclusion is that the test suite achieves **complete MC/DC coverage of all reachable logical paths**, supported by a line coverage of 97.9% and a branch coverage of 93.7%.